electronics

MDPI

Article

Comparative Evaluation of Perceptual Hashing and Deep Embedding Methods for Robust and Efficient Image Deduplication

Md Firoz Mahmud †, Zerin Nusrat † and W. David Pan *

Department of Electrical and Computer Engineering, University of Alabama in Huntsville, Huntsville, AL 35899, USA

* Correspondence: pand@uah.edu

† These authors contributed equally to this work.

Abstract

The rapid growth in large-scale image repositories over the past few years has made exact and near-duplicate images increasingly common, creating substantial redundancy that wastes storage resources and reduces retrieval efficiency in practical systems. Even though perceptual hashing and deep learning are promising deduplication strategies, the lack of standardized benchmarks complicates direct comparison. In this study, we conduct a unified, controlled evaluation of five commonly used methods, including four classical perceptual hashes (AHash, DHash, PHash, and WHash) and a CNN-based embedding model. We evaluate all methods on the UKBench and Amazon Berkeley Objects datasets using identical preprocessing, thresholds, and metrics, which include exact duplicates, near-duplicates, and geometrically transformed duplicates. Our experiments highlight a clear trade-off between speed and robustness. Hashing methods are computationally efficient and effective for exact matches, but perform poorly on near-duplicates and under geometric transformations, whereas the CNN model is significantly more robust across all duplicate types, but comes at a high computational cost. Based on these results, we outline practical recommendations for selecting deduplication strategies in large-scale applications. In addition, our evaluation setup serves as a reproducible baseline for future research in image similarity and large-scale deduplication.

Keywords: image deduplication; perceptual hashing; deep learning; convolutional neural networks; image similarity; empirical evaluation

Check for updates

Academic Editor: Kefeng Ji

Received: 13 February 2026
Revised: 26 March 2026
Accepted: 30 March 2026
Published: 2 April 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

The volume of digital data generated worldwide is increasing at an unprecedented pace, due to the rise of cloud computing, social media, online commerce, Internet of Things (IoT) technologies, and large-scale multimedia applications [1–3]. Industry forecasts suggest that the global datasphere was estimated to be around 149 zettabytes (10^{21} bytes) in 2024, being expected to surpass 180 zettabytes by the end of 2025. If the same growth trend continues, 2026 could see data volumes reach roughly 230–240 zettabytes [4–6]. With this continuous surge in data, keeping storage cost-effective has become one of the most urgent challenges for today's data centers and cloud storage systems, as reflected both in architectural discussions of warehouse-scale computing [7] and in recent studies on cloud storage cost complexity [8].

Large-scale workload measurements reported by major industry players, including Microsoft, Google, and enterprise storage vendors, show that redundancy is common across primary and secondary storage systems, with practical deployments frequently exceeding 50% redundant data [9–11]. This redundancy not only increases storage demand, but also slows down retrieval efficiency, and negatively affects downstream applications such as image search, classification, clustering, and recommendation [12–14]. Consequently, data deduplication has become a widely adopted strategy in production storage systems. Industry reports suggest widespread enterprise adoption of data deduplication to lower storage overhead and operating costs [15], whereas academic evaluations emphasize its significance to boost storage efficiency [16].

This study focuses on large-scale image deduplication methods. Early research on image deduplication mostly relied on perceptual hashing techniques. Venkatesan et al. [17] proposed a robust hashing framework that converts an image into a compact binary code, enabling efficient similarity comparison while remaining stable under common image processing operations such as compression and small photometric changes. In [18], the authors used multi-resolution models from the Haar transform to derive feature vectors from images. They proved that the energy distribution of these transformation coefficients across various scales successfully captures the features of images for content-based image retrieval. Zauner's benchmark study [19] evaluated various perceptual hashing algorithms that compare low-frequency discrete cosine transform (DCT) coefficients to their median value to create a compact binary hash for precise image similarity comparison. Due to the low computational complexity and minimal memory footprint, perceptual hashing approaches (AHash, DHash, PHash, and WHash) have been widely adopted for detecting exact duplicates and near-duplicates under mild photometric variations [20]. In addition to proposing specific hashing algorithms, several studies have provided broader analyses and applications of perceptual hashing techniques. Monga and Evans [21] provided one of the earlier surveys on perceptual image hashing, outlining core design principles and robustness-efficiency trade-offs. In [22], Swaminathan et al. proposed a strong and secure image hashing approach. Their method generates hashes that withstand regular signal processing operations such as compression, filtering, and minimal geometric distortions, while secret key randomization protects against malicious attacks. In addition, Khelifi and Jiang [23] proposed a perceptual hashing method based on virtual watermark detection. The technique produces reliable hash values that maintain perceptual similarity by making use of the detector reaction to pseudo-random watermark patterns inside a decision-theoretic framework, thereby supporting the application of perceptual hashing for image similarity and duplicate detection tasks.

Despite being efficient and popular, several studies have reported that perceptual hashing methods exhibit limited robustness when images are affected by geometric transformations. In a recent study [24], Sharma et al. highlighted this limitation by examining distance distributions. They showed that common hashing methods fail to match images modified by basic geometric changes such as rotation, scaling, and cropping. In [25], McKeown et al. demonstrate that perceptual hashes struggle with spatial consistency; specifically, they show that shifting an object's layout often results in unreliable similarity scores. A consistent trend was also observed by Kotzer et al. [26], where they found that standard perceptual hashes were insufficient for modern image editing and that neural network-based features performed better. These findings suggest that classical hashing methods often do not perform well in real-world conditions, where duplicate images are often altered by different angles, recompression, editing, and background clutter.

Observing the limitations of perceptual hashing methods under geometric transformations, several recent studies focused on deep learning-based features for image similarity

and deduplication. Deep convolutional neural networks (CNNs) can learn sophisticated, hierarchical visual representations, as demonstrated by models such as AlexNet [27] and VGG [28]. Additional study found that these learned embeddings capture high-level semantic details and are inherently more resilient to geometric and visual variations than handcrafted features, making them superior for matching tasks [29]. To this end, Xia et al. [30] introduced deep feature learning strategies designed for near-duplicate detection and demonstrated that they were significantly more robust than classical hashing methods. This advantage is supported by recent domain-specific benchmarks. For instance, Truong et al. [31] ran an extensive test on medical image datasets and found that CNN-based embeddings are much more reliable than classical hashing methods when dealing with real-world issues like structural changes or varied imaging conditions.

To address the trade-off between robustness and efficiency, recent studies have started looking into hybrid and transformer-based models. Jakhar et al. [32] incorporated deep learning features into a perceptual hashing framework to improve near-duplicate detection with geometric distortions at a moderate computational cost. At the same time, transformer architectures have become a potent method of learning highly contextual and invariant representations. These architectures are based on the self-attention mechanism as presented by Vaswani et al. [33]. Chen et al. [34], for instance, used this mechanism to create a transformer-based hashing technique. Recent developments, like the work of Mahmud et al. [35], show that attention is still relevant for producing effective, compact representations by optimizing its use for particular tasks like lossless compression. Despite their strengths, learning-based techniques generally struggle with scalability due to their high computational overhead compared to simpler perceptual hashing methods, which can create scalability bottlenecks in large-scale systems [36].

Nonetheless, the absence of consistent benchmarks makes it difficult to compare a large collections of hashing techniques objectively. Fair, direct comparisons between traditional and deep learning approaches are challenging because researchers use heterogeneous evaluation settings, which include different datasets, preprocessing steps, and performance metrics [31]. Furthermore, many studies only assess performance on one type of duplicate (e.g., exact copies), failing to jointly evaluate the entire spectrum of adversarially transformed copies, near-duplicates, and exact duplicates that are typical of real-world image collections.

To overcome these challenges, in this paper, we proposed a unified evaluation framework that allows for a fair and systematic comparison between traditional perceptual hashing and more recent deep learning-based deduplication techniques. Analyzing all three duplicate types under a single framework allows us to demonstrate how different design decisions influence an algorithm's accuracy and efficiency in real-world scenarios. This analysis provides a roadmap for navigating the practical challenges of deduplication. This comparative study also helps determine exactly when simple hashing is enough and when it is time to switch to more complex, learning-based models.

2. Materials and Methods

In this section, we outline the experimental setup and evaluation procedure used to test various image deduplication methods under consistent, reproducible conditions. The objective is to examine how different algorithms respond to increasing visual diversity, ranging from exact duplicates to substantially altered versions of the original image. Five popular techniques are included in our selection, which is split into two groups: traditional perceptual hashing techniques (AHash, DHash, PHash, and WHash) and a deep learning technique based on CNN embeddings. These particular hashing algorithms were selected as typical examples of effective, manually created descriptors that are frequently

benchmarked in the literature [19,20]. This choice allows robustness-oriented feature learning and efficiency-oriented hashing to be compared side by side.

Evaluation was conducted using benchmark image collections derived from the UK-Bench (UKB) and Amazon Berkeley Objects (ABO) datasets. We produced three subsets for each dataset that were intended to mimic real-world duplication: exact copies, near duplicates with slight photometric changes, and transformed duplicates featuring geometric shifts like cropping, flipping, and rotation. To ensure consistency, we followed a four-step pipeline: (i) building duplicate-specific datasets, (ii) extracting image representations using either hashing or CNN embeddings, (iii) computing similarities with the appropriate distance measure for each method, and (iv) evaluating performance quantitatively. Our evaluation concentrates on four key areas: retrieval precision, ranking effectiveness, set overlap, and speed. For accuracy, we use Precision–Recall and ROC (Receiver Operating Characteristic) analysis; for ranking quality, we use MAP (Mean Average Precision) and NDCG (Normalized Discounted Cumulative Gain); for set comparisons, we use Jaccard similarity; runtime data offers insights into computational efficiency. A general overview of the data deduplication process is illustrated in Figure 1.

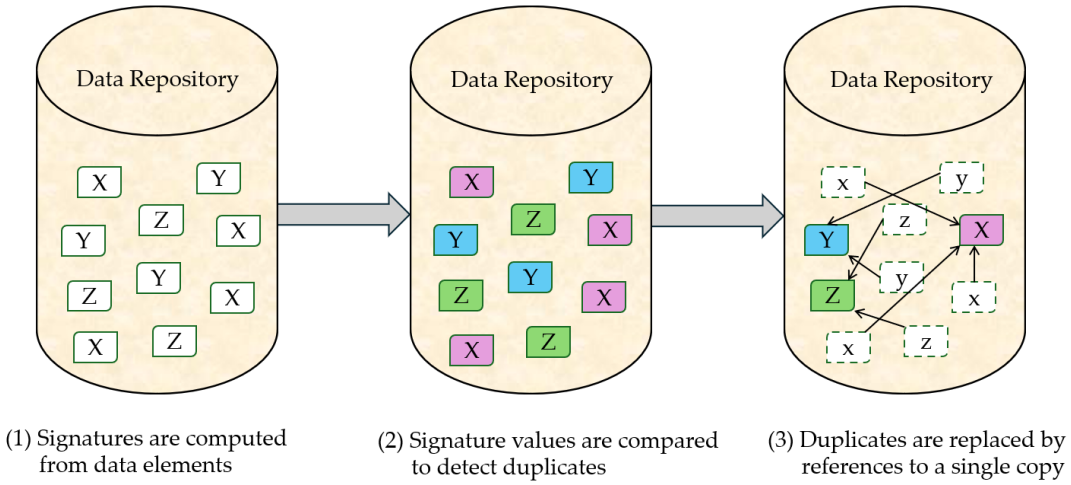


Figure 1. Data deduplication process.

The following subsections will discuss how we set up our study. This includes the techniques we chose, our way of extracting features, the math behind our comparisons, and the data and metrics we used to measure the overall performance.

2.1. Overview of the Methods Evaluated

To benchmark classical hashing and deep-learning-based representations within a a single evaluation setup, we included four perceptual hashing approaches (AHash, DHash, PHash, and WHash), as well as a pretrained CNN embedding approach.

2.1.1. Average Hash (AHash)

Among perceptual hashing techniques, Average Hash (AHash) is commonly used in perceptual hashing-based deduplication [37] and is widely used as a lightweight baseline in comparative studies [38]. AHash is computationally cheap and deterministic for any input by reducing an image's global luminance structure to a compact 64-bit binary descriptor through its fixed, non-learning pipeline of resizing, grayscaling, and mean-based thresholding [39].

The algorithm is depicted in Figure 2 and the processing steps are summarized as follows:

- Resizing the image

We first normalize the input image by resizing it to a fixed $n \times n$ grid (typically 8×8), which aggressively down-samples high-frequency details while preserving coarse structures. This ensures the hash is unaffected by changes in the original image dimensions.

- Converting to grayscale: Following the resizing step, the image undergoes a grayscale conversion to consolidate three-channel RGB data into a single luminance channel. This reduction effectively removes redundancy while isolating the intensity information most critical for perceptual comparison [19]. This results in an 8×8 matrix containing intensity values I_i.

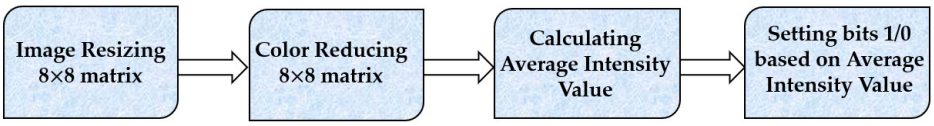


Figure 2. AHash computation pipeline.

- Computing the Average Intensity: To set a single global threshold, AHash computes the mean intensity over all $M = 64$ pixels in the resized image, as per its standard formulation [39]. The mean luminance is determined using the following equation:

$$\mu_I = \frac{1}{M} \sum_{i=1}^{M} I_i, \tag{1}$$

where I_i is the grayscale intensity of the ith pixel.

- Generating the binary hash: After computing the mean intensity, the algorithm thresholds the grayscale matrix pixel by pixel. If a pixel's intensity is greater than or equal to the average, the bit is set to 1, or to 0 otherwise. Arranging these bits in row-major order produces a 64-bit descriptor that captures the overall luminance pattern of the image.

Although AHash is extremely fast and computationally lightweight, its dependence on a global intensity pattern makes it sensitive to illumination changes, contrast shifts, and geometric transformations [21]. Nonetheless, due to its low complexity and high speed, AHash remains valuable as a baseline method in duplicate detection pipelines and is often used as an initial screening step to filter obvious duplicates before applying stronger hashing methods or deep learning models.

2.1.2. Difference Hash (DHash)

Difference Hash (DHash) focuses on image structure by comparing neighboring pixels and encoding their relative intensity differences [19]. Unlike AHash, which uses a single global mean as a threshold, DHash encodes local brightness transitions (gradients), which generally makes it less sensitive to illumination changes. The algorithm proceeds through the following steps as shown in Figure 3:

- Resizing the image: DHash begins by resizing the image to $(n+1) \times n$ (commonly 9×8). Adding one extra column makes it possible to compare horizontal pixel pairs within each row. This resizing suppresses high-frequency content and ensures consistent dimensions across all images.
- Converting to grayscale: The resized image is then mapped to a grayscale representation, yielding a 9×8 matrix of luminance values $I(i, j)$. This lowers computation because only one channel is processed.

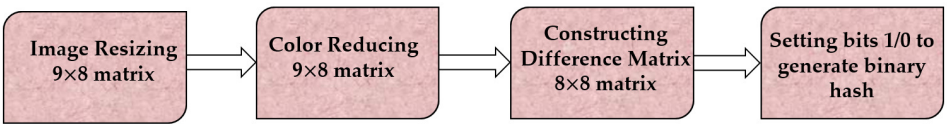


Figure 3. DHash computation pipeline.

- Computing horizontal differences: At its core, DHash represents an image by comparing the intensities of horizontally neighboring pixels and encoding their relative differences [19]. Within each row, the algorithm performs pairwise comparisons between each pixel and the pixel immediately to its right. Let $D(i,j)$ denote the binary difference field produced by these comparisons:

$$D(i,j) = \begin{cases} 1, & \text{if } I(i,j) > I(i,j+1); \\ 0, & \text{otherwise.} \end{cases} \tag{2}$$

  This step turns the 9×8 matrix into an 8×8 gradient matrix that captures structural transitions.
- Generating the binary hash: Next, each entry in the 8×8 difference field is converted into a binary bit. A value of 1 is assigned when the left pixel intensity exceeds the right pixel intensity; otherwise, a value of 0 is assigned. The specific ordering of these comparisons does not affect the hash as long as it is kept consistent across images. The values of $D(i,j)$ are concatenated row-wise to form a 64-bit DHash fingerprint, representing the dominant horizontal gradient pattern of the image.

We can see that DHash is more resistant to illumination shifts than AHash, because DHash is based on relative intensity between neighboring pixels, which tends to remain stable even when the brightness shifts uniformly [19]. DHash strikes a practical balance between computational efficiency and structural encoding, making it suitable for detecting nearly identical images [13]. However, under geometric transformations, DHash degrades because operations such as rotation, cropping, and perspective shifts change pixel adjacency and distort the underlying gradient relationships [21]. So, it performs best when the image layout does not change much.

2.1.3. Perceptual Hash (PHash)

Unlike AHash and DHash, which operate in the spatial domain, Perceptual Hash (PHash) works in the frequency domain by applying the Discrete Cosine Transform (DCT) to break the image into frequency components. It then relies on low-frequency DCT coefficients, which tend to remain stable under compression, mild rotations, and illumination changes, giving PHash stronger robustness than spatial-domain hashes [19]. The algorithm proceeds through the following steps:

- Resizing and grayscale conversion: In the beginning, the image is normalized to a fixed size of 32×32, and then mapped to grayscale. This results in a consistent luminance representation that supports DCT-based frequency analysis while retaining the necessary structure for perceptual hashing.
- Applying the 2D Discrete Cosine Transform (DCT): A two-dimensional DCT is used on the grayscale image $I(i,j)$ to convert it into a frequency-domain representation $F(u,v)$. The DCT formula utilized is the standard type-II 2D DCT [40]:

$$F(u,v) = \frac{1}{4} C_u C_v \sum_{i=0}^{M-1} \sum_{j=0}^{M-1} I(i,j) \cos\left[\frac{(2i+1)u\pi}{2M}\right] \cos\left[\frac{(2j+1)v\pi}{2M}\right] \tag{3}$$

where C_u and C_v are normalization factors defined as:

$$C_u = \begin{cases} \frac{1}{\sqrt{2}}, & u = 0, \\ 1, & u > 0. \end{cases} \qquad C_v = \begin{cases} \frac{1}{\sqrt{2}}, & v = 0, \\ 1, & v > 0. \end{cases}$$

- Extracting the 8 × 8 low-frequency block: To capture global structure, PHash retains only low-frequency DCT coefficients while discarding higher-frequency detail. In practice, the top-left 8 × 8 block of the DCT matrix is extracted to create a compact feature set that can withstand minor image changes.
- Median thresholding and hash generation: The median of the 64 values in the low-frequency block is calculated as:

$$\eta = \text{median}(F_{8\times 8}) \tag{4}$$

A 64-bit hash is then created by comparing each coefficient to this median:

$$H(i,j) = \begin{cases} 1, & F(i,j) \geq \eta; \\ 0, & F(i,j) < \eta. \end{cases} \tag{5}$$

Concatenating these values row-wise yields the final PHash fingerprint.
- Comparing hashes using Hamming distance: Similarity between two images is calculated using the Hamming distance between their hash vectors H_1 and H_2:

$$d(H_1, H_2) = \sum_{i=1}^{m} (H_{1i} \oplus H_{2i}) \tag{6}$$

where $\oplus$ denotes the XOR operation. A smaller Hamming distance indicates greater similarity between images. This comparison method is standard for perceptual hash fingerprints [19].

PHash exhibits strong resilience to changes in illumination and to JPEG compression [41,42]. The JPEG standard itself preserves low-frequency DCT coefficients, which PHash leverages to maintain hash stability [19]. Thus, it provides significantly better discriminative capability than AHash and DHash. Large rotations, heavy cropping, or significant distortions that change the global frequency structure cause it to perform worse [43]. Additionally, PHash requires more processing power than basic spatial hashing techniques.

2.1.4. Wavelet Hash (WHash)

Wavelet Hash (WHash) uses a 2D Haar wavelet transform to create a perceptual fingerprint that captures the structure of the image at various resolutions [18,44]. While PHash depends on DCT frequency coefficients, and AHash and DHash operate directly in the spatial domain, WHash employs wavelet decomposition to capture both spatial layout and frequency content. Robustness to minor geometric and photometric distortions is usually enhanced by this mixed representation [17]. The foundation of our WHash implementation is the imagededup library [45]. In this method, the Haar wavelet transform's approximation coefficients are used to generate the hash. Below is the processing flow:

- Resizing and grayscale conversion: The image is first converted to grayscale and resized (typically to 64 × 64). Because of this uniform input, the wavelet decomposition is consistent for every image.
- Applying the 2D Haar wavelet transform: Next, a separable two-dimensional Haar wavelet transform is applied to the grayscale image $I(x, y)$. One approximation

sub-band, A, and three detail sub-bands, H, V, and D, which capture horizontal, vertical, and diagonal variations, respectively, are produced by this decomposition. The transformation can be expressed as:

$$(A, H, V, D) = \text{Haar2D}(I) \tag{7}$$

where A represents the low-frequency (coarse) content of the image, and the detail sub-bands capture the high-frequency variations.

- Selecting approximation coefficients: We use approximation coefficients A, which are stable under minor changes like illumination shifts, compression, or blur. The resulting coefficient set is then used to generate the perceptual hash.
- Median thresholding and hash generation: Next, we compute the median of the approximation coefficients and denote it by τ

$$\tau = \text{median}(A). \tag{8}$$

Each coefficient is compared to the threshold to produce a binary value:

$$H_i = \begin{cases} 1, & A_i \geq \tau; \\ 0, & A_i < \tau. \end{cases} \tag{9}$$

Finally, the thresholded bits are concatenated to form the 64-bit WHash fingerprint.

WHash uses wavelet features to retain information from both the spatial and frequency domains, improving robustness to blur, filtering, and small rotations compared to AHash and DHash [17]. At the same time, it maintains a favorable balance of computational cost and perceptual stability. Despite its improved stability, WHash can degrade under large geometric transformations, as major rotations and heavy cropping disrupt the underlying multi-resolution representation [23]. It has a longer runtime than basic spatial hashes, but it is still much less expensive computationally than deep learning methods.

2.1.5. Convolutional Neural Network (CNN)-Based Embedding Model

The CNN-based method runs images through a pretrained deep convolutional network to obtain high-dimensional feature vectors, leveraging hierarchical feature abstractions learned from large-scale image datasets [28,46]. VGG-16 was selected as the representative CNN baseline because it is a gold-standard architecture for robust feature extraction in image retrieval literature. In contrast to hashing methods that use handcrafted pixel or frequency-based descriptors, CNN embeddings are learned features that represent higher-level semantics like shapes, textures, and object structure. These learned representations are more robust than traditional approaches for detecting duplicates, as they remain stable under a wider range of image transformations [26,30].

- Preprocessing and resizing: Before feature extraction, images are resized to fit the CNN backbone's input size (usually 224×224) and normalized using ImageNet mean and variance. This standard preprocessing ensures the extracted feature vectors are consistent and stable.
- Deep feature extraction: We use a pretrained VGG-16 network [28,47] as a fixed feature extractor. The classification head (fully connected layers) is discarded, leaving only the convolutional backbone to generate feature maps. The convolutional base generates a deep feature tensor for an input image (I), which is then converted into a fixed-length embedding using global average pooling:

$$z = \text{GAP}(\text{CNN}_{\text{conv}}(I)), \tag{10}$$

where $\mathbf{z} \in \mathbb{R}^d$ (with d typically in the thousands) and Global average pooling (GAP) is used to convert the convolutional feature maps into a fixed-length vector z, following the formula introduced in Network-in-Network model [48]. These embeddings measure semantic similarity rather than pixel-level similarity [46,49].

- Similarity computation using cosine similarity: Cosine similarity, a common metric for comparing commercial CNN features, is used to measure the similarity between two deep feature embeddings $\mathbf{z}_1$ and $\mathbf{z}_2$ [50]:

$$\text{sim}(\mathbf{z}_1, \mathbf{z}_2) = \frac{\mathbf{z}_1 \cdot \mathbf{z}_2}{|\mathbf{z}_1||\mathbf{z}_2|}. \tag{11}$$

This metric yields values in $[-1, 1]$ and measures the angular distance between vectors. Values close to 1 indicate high semantic similarity and possible duplicates for normalized non-negative embeddings (common with ReLU-based CNNs), whereas lower values indicate dissimilarity. Cosine similarity is widely used in deep feature-based image retrieval and duplicate detection [30,46]. The cosine similarity score is thresholded to determine the final duplicate/non-duplicate decision.

2.2. Feature Extraction Pipeline

Using the algorithmic definitions in Section 2.1, we describe the feature-generation procedure used in our experiments. For each image in the Exact Duplicate, Near Duplicate, and Transformed datasets, feature descriptors were extracted using all five approaches: AHash, DHash, PHash, WHash, and the CNN-based embedding model.

2.2.1. Hash-Based Feature Generation

Each image was encoded as a 64-bit binary descriptor using the four perceptual hashing techniques (AHash, DHash, PHash, and WHash), as described in Section 2.1. The similarity between image pairs was determined using the Hamming distance, making it possible to compare these small fingerprints effectively. The Hamming distance between two hash vectors was transformed into a normalized similarity score in the interval $[0, 1]$ in order to facilitate score-based evaluation for Precision–Recall (PR), ROC, and F1-score curves which is shown in Equation (12).

$$s_{\text{hash}} = 1 - \frac{d_H(H_1, H_2)}{64} \tag{12}$$

where $d_H(H_1, H_2)$ is the Hamming distance between two 64-bit hash descriptors. For tabulated results, we additionally assessed performance at three fixed Hamming distance thresholds (0, 10, and 32), which correspond to strict, moderate, and relaxed matching conditions. The entire hash-based feature extraction and similarity comparison procedure is shown in Figure 4.

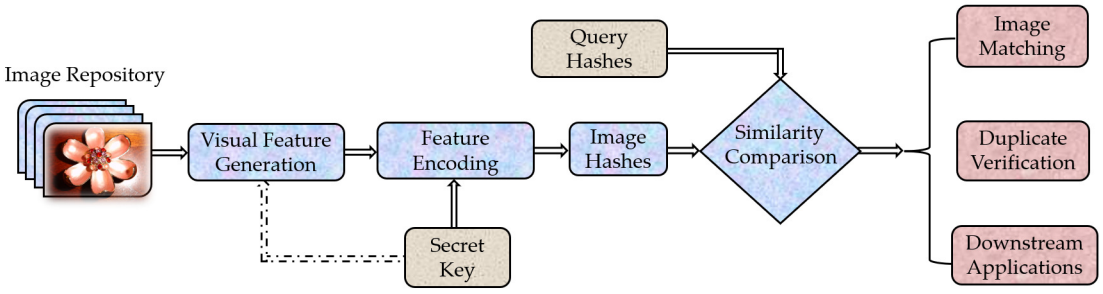


Figure 4. Hash-based feature generation and similarity comparison pipeline for perceptual hashing methods.

2.2.2. CNN-Based Feature Generation

Before being run through a pretrained CNN, images were resized and normalized to produce high-dimensional embedding vectors (see Section 2.1). Cosine similarity, which generates continuous similarity scores in the interval $[-1, 1]$, was used to calculate the similarity between embedding vectors. For score-based Precision–Recall (PR), ROC, and F1-score evaluation, these raw cosine similarity scores were directly utilized.

Three fixed cosine similarity thresholds (0.5, 0.9, and 1.0) were used to further analyze performance under relaxed, moderate, and strict matching conditions for tabulated results. The entire CNN-based feature extraction and matching pipeline is shown in Figure 5.

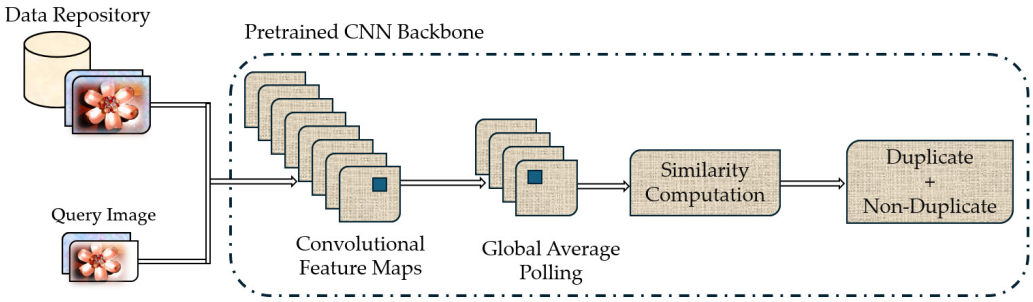


Figure 5. CNN-based deep feature extraction and similarity comparison pipeline used for image deduplication.

2.2.3. Evaluation Strategy

The extracted hash descriptors and CNN embeddings were used to calculate the Precision–Recall (PR) curves, ROC curves, F1-scores, MAP, NDCG, Jaccard similarity, and runtime, as reported in Section 3.

Two complementary evaluation strategies were used because different metrics require different evaluation principles. Accuracy and ranking-based metrics such as Jaccard similarity, MAP, and NDCG were computed using fixed, method-specific thresholds, which is common practice in image deduplication. In contrast, PR, ROC, and F1-score curves were obtained using score-based evaluation by sweeping over normalized similarity scores, enabling threshold-independent evaluation. This separation allows fair comparison for tabulated results while providing a clear view of ranking behavior through curve-based analysis.

2.3. Evaluation Metrics

Multiple evaluation metrics were used to objectively evaluate the performance of the five deduplication methods across the Exact Duplicate, Near Duplicate, and Transformed datasets. These metrics measure accuracy, ranking quality, robustness, and computational efficiency at various similarity thresholds. The definitions of the evaluation metrics used in this study are provided in the following subsections.

2.3.1. Precision and Recall

In this study, precision is defined as the fraction of retrieved items that are true duplicates, whereas recall is the fraction of all true duplicates that the method retrieves. These measures are the standard for information retrieval and classification [51,52], including related tasks like copy detection [53]. Equations (13) and (14) calculate precision and recall, respectively.

$$\text{Precision} = \frac{TrPos}{TrPos + FlPos} \tag{13}$$

$$\text{Recall} = \frac{TrPos}{TrPos + FlNeg} \tag{14}$$

where $TrPos$, $FlPos$, and $FlNeg$ denote true positives, false positives, and false negatives, respectively. Precision–Recall (PR) curves depict the trade-off between these two quantities at various thresholds.

2.3.2. ROC Components: TPR and FPR

The Receiver Operating Characteristic (ROC) curve summarizes binary classification behavior by showing how the True Positive Rate (TPR) varies with the False Positive Rate (FPR) [54,55]. These quantities are defined as:

$$TPR = \frac{TrPos}{TrPos + FlNeg} \qquad FPR = \frac{FlPos}{FlPos + TrNeg} \tag{15}$$

where $TrNeg$ denotes true negatives.

2.3.3. F1-Score

The F1-score summarizes performance in a single value by calculating the harmonic mean of precision and recall [53,56]. It is defined as:

$$F_1 = 2\left(\frac{\text{Pr} \times \text{Rc}}{\text{Pr} + \text{Rc}}\right) \tag{16}$$

where Pr and Rc denote Precision and Recall, respectively.

2.3.4. Mean Average Precision (MAP)

Mean Average Precision (MAP) assesses the level of accuracy of a ranked list of results by calculating the average precision at each location where a relevant item is retrieved. It has become the standard metric for evaluating ranking performance in information retrieval [51], and its estimation properties are well-studied [57]. It is defined as:

$$MAP = \frac{1}{Q}\sum_{q=1}^{Q} AvgP(q) \tag{17}$$

where Q denotes the total number of queries, and $AvgP(q)$ represents the Average Precision for the qth query, defined as:

$$AvgP(q) = \frac{1}{R_q}\sum_{i=1}^{n} P(i) \cdot rel(i). \tag{18}$$

In this definition, R_q represents the number of relevant duplicates for the query q, $P(i)$ is the precision computed at rank i, and $rel(i)$ is an indicator function that specifies whether the image at rank i is relevant.

2.3.5. Normalized Discounted Cumulative Gain (NDCG)

NDCG is a standard ranking metric that prioritizes relevant duplicates retrieved at higher ranks, making it ideal for assessing ordered retrieval lists in modern information retrieval systems [51,58]. The notation "@i" evaluates the metric on only the top-i retrieved images, emphasizing the quality of the highest-ranked results.

$$NDCG@i = \frac{DCG@i}{IDCG@i} \tag{19}$$

where the Discounted Cumulative Gain (DCG) at rank i is computed as:

$$DCG@i = \sum_{k=1}^{i} \frac{2^{rel_k} - 1}{\log_2(k+1)} \tag{20}$$

here, rel_k denotes the relevance of the item at rank k, where $rel_k = 1$ for a true duplicate and $rel_k = 0$ otherwise. The ideal DCG (IDCG) is computed using the same expression as DCG, but with an ideal ordering, all relevant images appear before any irrelevant images [59,60].

2.3.6. Jaccard Similarity

The Jaccard similarity index measures the overlap between the predicted duplicate set (P) and the ground-truth set (Q). Originally proposed for set comparison in ecology, it has since become widely used in data mining applications [61,62]. It is defined as:

$$Jaccard(P, Q) = \frac{|P \cap Q|}{|P \cup Q|}, \tag{21}$$

which represents the ratio between the number of elements common to both sets, and the number of total unique elements in either set.

2.3.7. Runtime Measurement

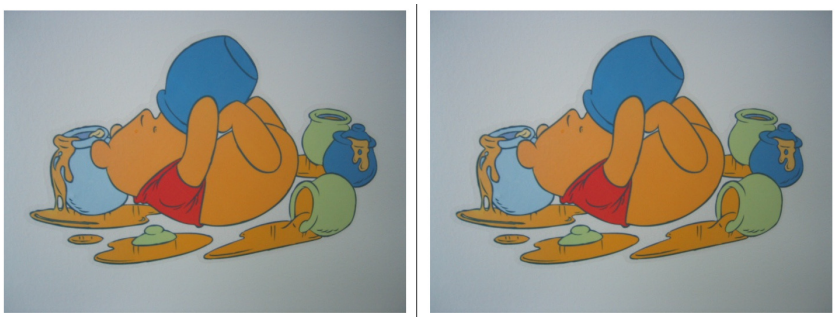
The total runtime was calculated as follows to assess computational efficiency:

$$T_{\text{total}} = T_{\text{encode}} + T_{\text{search}}. \tag{22}$$

In this notation, T_{encode} is the per-image feature extraction time, while T_{search} represents the time spent computing similarity over the dataset. Runtime is commonly reported alongside accuracy-based measures in large-scale retrieval and duplicate detection studies because scalability and computational cost are critical in real-world deployment [63–65].

2.4. Dataset Description

Two benchmark datasets were used in this study: the UKBench (UKB) dataset [66] and the Amazon Berkeley Objects (ABO) dataset [67]. For each dataset, we created three subsets corresponding to Exact Duplicate, Near Duplicate, and Transformed cases. Sample images from UKB and ABO are shown in Figures 6 and 7 respectively.



(**a**) Exact duplicates (EDs)



(**b**) Near duplicates (NDs)

Figure 6. *Cont.*



(**c**) Transformed duplicates (TDs)

Figure 6. Sample images from the UKB dataset illustrating different duplicate scenarios: (**a**) exact duplicates (EDs), (**b**) near duplicates (NDs) with minor photometric variations, and (**c**) transformed duplicates (TDs) involving geometric transformations such as rotation and viewpoint change.



(**a**) Exact duplicates (EDs)



(**b**) Near duplicates (NDs)



(**c**) Transformed duplicates (TDs)

Figure 7. Sample images from the ABO dataset illustrating different duplicate scenarios: (**a**) exact duplicates (EDs), (**b**) near duplicates (NDs) with minor photometric variations, and (**c**) transformed duplicates (TDs) involving geometric transformations such as rotation and viewpoint change.

2.4.1. UKB Dataset

The UKB dataset [66] was created for image retrieval and includes 10,200 images of 2550 objects. Each object is photographed four times, typically from different angles and occasionally with minor lighting changes. Because of this four-image grouping, UKB automatically generates near-duplicate sets and has become a standard benchmark in content-based image retrieval.

This work used the following subsets for large-scale quantitative evaluation:

- Exact Duplicate: 5100 images
- Near Duplicate: 10,200 images
- Transformed: 10,200 images

All tabulated results were computed on the full Exact Duplicate, Near-Duplicate, and Transformed subsets, whereas for graphical analysis, including Precision–Recall (PR), ROC, and F1-score curves, a curated subset of 1000 Near-Duplicate (ND) images and 1002 Transformed-Duplicate (TD) images from the UKB dataset was used (Figure 6). The selected images form groups of visually similar instances captured from different viewpoints and subjected to geometric variations such as flipping, rotation, cropping, and changes in aspect ratio.

2.4.2. ABO Dataset

The Amazon Berkeley Objects (ABO) dataset [67] is composed of actual Amazon.com product listings. The catalog includes 147,702 products, 398,212 unique images, 360-degree turntable sequences, and artist-created 3D models for thousands of items. Because each product contains extensive metadata (such as category, brand, and physical attributes), ABO serves as a large-scale benchmark for object understanding and retrieval.

This study only used 2D catalog images. The subsets constructed for quantitative evaluation include:

- Exact Duplicate: 2000 images;
- Near Duplicate: 3000 images;
- Transformed: 3000 images.

To demonstrate PR, ROC, and F1-score behavior, we chose 660 representative images from the Near-Duplicate (ND) and Transformed-Duplicate (TD) subsets. Each ND group contains three images with similar visual content, whereas each TD group contains five variations of the same image, such as flipped, resized, and rotated versions. These transformed samples were created using the same augmentation scheme as in the UKB experiments, but without cropping. This option preserves the underlying dataset structure while allowing for clear visual comparison across duplicate categories.

2.5. Experimental Setup

We conducted all experiments in Python 3.10 on a dedicated workstation using JupyterLab. All methods were evaluated under identical hardware and software conditions to ensure fairness. The system included an Intel® Core™ i7-5930K processor at 3.50 GHz, 64 GB RAM, and an NVIDIA GeForce GTX TITAN X GPU (12 GB) running Ubuntu 22.04 LTS. In this environment, we completed all pipeline steps, including dataset loading, descriptor generation for each method, duplicate-search operations, and metric computation. The runtime values reported in Section 3 were measured on the same hardware configuration. Using a single, consistent setup improves reproducibility and comparability of all results.

3. Results and Analysis

3.1. Graphical Evaluation on the UKB Dataset

This section presents the graphical performance comparison of the five methods on the UKB dataset. PR, ROC, and F1-score plots were created using a curated set of 1000 Near-Duplicate (ND) images and 1002 Transformed-Duplicate (TD) images. For perceptual hashing methods, the curves were generated by sweeping similarity scores normalized from Hamming distance over the range $[0, 1]$. In contrast, the CNN-based approach used cosine similarity scores swept over the full range $[-1, 1]$ for score-based evaluation.

3.2. Precision–Recall Curve Analysis (UKB)

Figure 8a presents the Precision–Recall (PR) curves for the UKB Near-Duplicate (ND) subset. All perceptual hashing methods show a sharp drop in precision as recall increases, suggesting that their effectiveness is limited once recall moves beyond very low levels. While PHash and DHash achieve high precision at the initial points, their performance declines rapidly, and AHash and WHash follow a similar trend with slightly slower but still notable decreases.

The same behavior is more pronounced for the Transformed-Duplicate (TD) subset in Figure 8b. Here, the PR curves of all hashing methods fall quickly at very low recall values, reflecting their sensitivity to geometric transformations. In contrast, the CNN-based method consistently maintains high precision over a broad range of recall in both subsets, with performance dropping only near full recall. This highlights the stronger robustness of learned deep features compared to handcrafted hash methods for image deduplication.

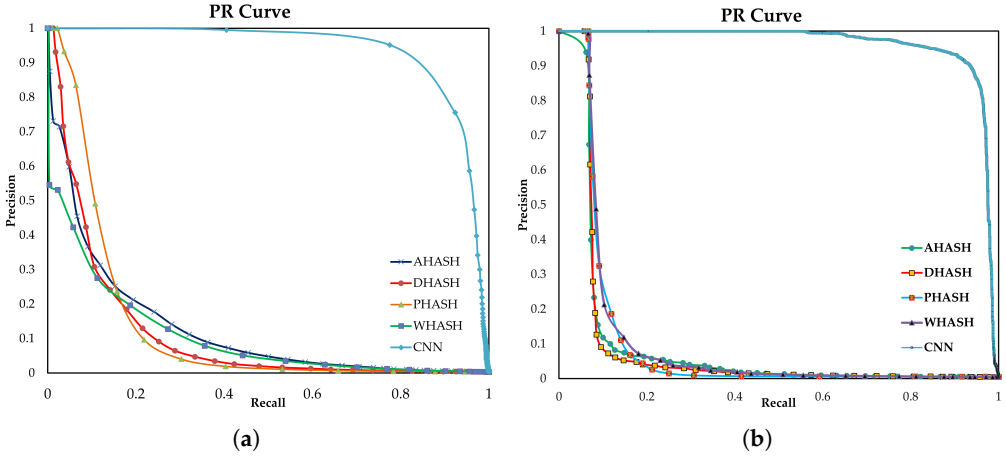


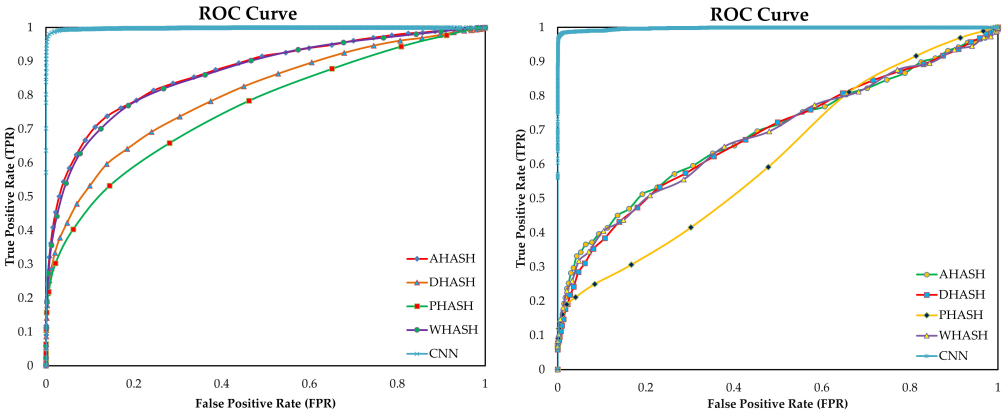
Figure 8. Precision–Recall curves for the UKB dataset: (**a**) Near Duplicate subset; (**b**) Transformed subset.

3.2.1. ROC Curve Analysis (UKB)

The ROC curve for the UKB Near-Duplicate (ND) subset is illustrated in Figure 9a. All perceptual hashing methods follow a similar pattern, with the true positive rate increasing gradually as the false positive rate rises. AHash and WHash perform slightly better at low false positive rates, while DHash and PHash lag behind by a small margin. Overall, the differences among the hashing methods are minor, indicating limited ability to clearly separate duplicates from non-duplicates.

The Transformed-Duplicate (TD) results in Figure 9b further emphasize this limitation. All hashing methods suffer a noticeable drop in performance under geometric transformations, with PHash showing the largest decline, especially at moderate false positive rates. In contrast, the CNN-based method achieves a near-ideal ROC curve in both subsets, rising sharply near the origin and maintaining a high true positive rate across the full range of

false positives. This confirms the strong and stable discriminative power of deep features, consistent with the behavior observed in the PR curves.



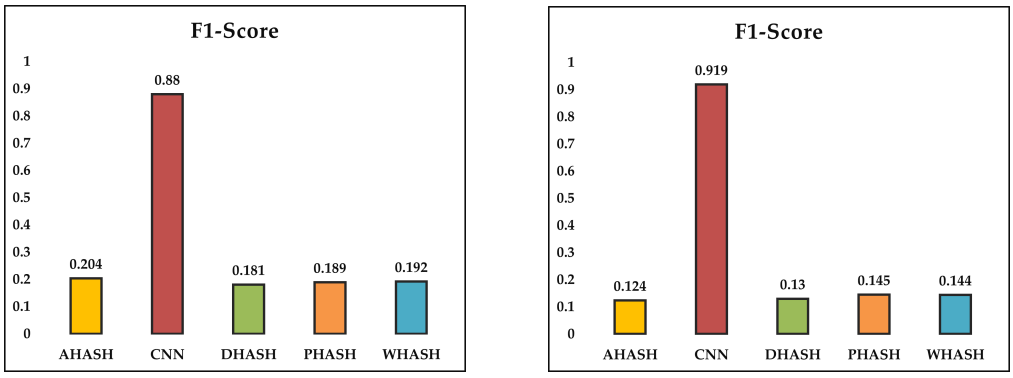
(**a**) (**b**)

Figure 9. ROC curves for the UKB dataset: (**a**) Near Duplicate subset; (**b**) Transformed subset.

3.2.2. F1-Score Analysis (UKB)

Figure 10 shows the F1-score results for the UKB dataset. For the Near-Duplicate (ND) subset (Figure 10a), all perceptual hashing methods produce low F1-scores, indicating a weak balance between precision and recall. Among the hashing techniques, AHash performs best, with WHash and PHash close behind, while DHash shows slightly lower performance. In contrast, the CNN-based method achieves a much higher F1-score of about 0.88, demonstrating a significantly better precision–recall balance.

A similar trend is observed for the Transformed-Duplicate (TD) subset in Figure 10b. The F1-scores of hashing methods remain low, with PHash and WHash performing slightly better than AHash and DHash, reflecting their limited robustness to geometric transformations. The CNN-based method further improves in this setting, reaching an F1-score of approximately 0.92, which highlights its strong robustness and consistent discriminative performance under more challenging visual variations.



(**a**) (**b**)

Figure 10. F1-score comparison for the UKB dataset using 100 randomly selected images: (**a**) Near Duplicate subset; (**b**) Transformed subset.

3.3. Graphical Evaluation on the ABO Dataset

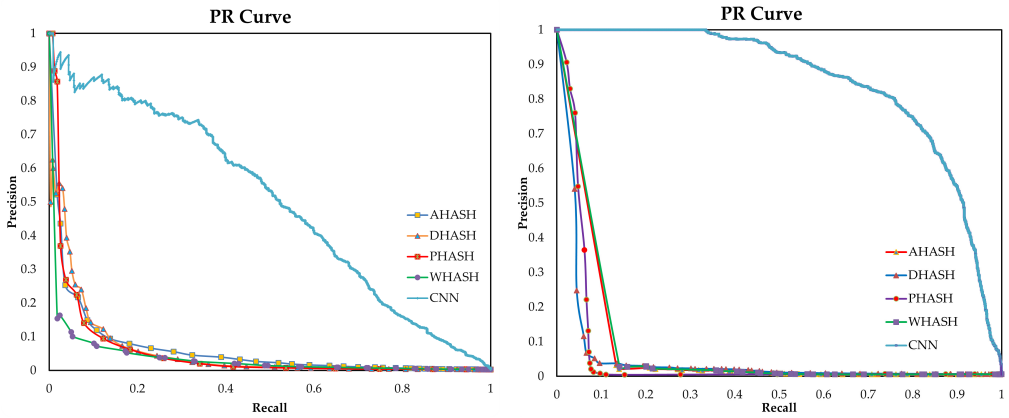
The ABO dataset presents a more challenging evaluation scenario than UKB due to higher intra-class variability and more diverse object appearances and backgrounds. To capture these effects, PR, ROC, and F1-score plots were created with a curated set of 660 images from the Near-Duplicate (ND) and Transformed-Duplicate (TD) subsets. The ND subset is made up of visually similar images, whereas the TD subset contains

multiple transformed versions of each image. These graphical results demonstrate how each method responds to increased appearance diversity and transformation complexity, as well as the performance gap between perceptual hashing techniques and CNN-based embeddings under more realistic deduplication conditions.

3.3.1. Precision–Recall Curve Analysis (ABO)

The Precision–Recall (PR) curve for the ABO Near-Duplicate (ND) subset is illustrated in Figure 11a. All perceptual hashing methods experience a rapid drop in precision with only a small increase in recall, indicating difficulty in handling the high visual diversity and complex backgrounds of ABO product images. While some hashing methods achieve reasonable precision at very low recall levels, their performance quickly degrades as more duplicate pairs are retrieved.

This trend becomes even more pronounced for the Transformed-Duplicate (TD) subset in Figure 11b. When images are subjected to transformations such as flipping, cropping, and rotation, the PR curves of all hashing methods collapse at very low recall values, revealing poor robustness to appearance changes. In contrast, the CNN-based method maintains much higher precision over a broad range of recall in both subsets, highlighting its stronger ability to capture semantic similarity in visually complex and transformed product images.



(**a**) (**b**)

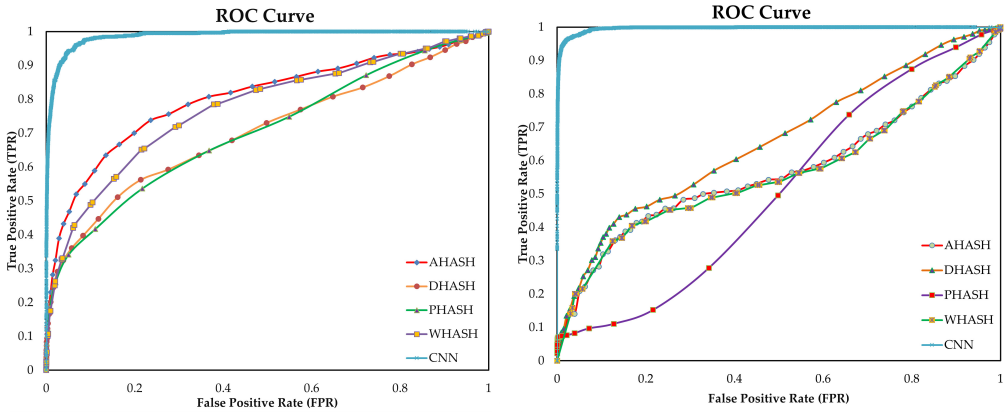
Figure 11. Precision–Recall curves for the ABO dataset: (**a**) Near Duplicate subset; (**b**) Transformed subset.

3.3.2. ROC Curve Analysis (ABO)

Figure 12a shows the ROC curves for the ABO Near-Duplicate subset. All four hashing methods display a gradual increase in true positive rate as the false positive rate rises, indicating limited but meaningful ability to distinguish duplicates under near-duplicate conditions. Among the hashing approaches, AHash and WHash perform slightly better, while PHash and DHash show weaker discrimination.

For the Transformed subset in Figure 12b, the performance of hashing methods declines further. The ROC curves become flatter, especially at low false positive rates, highlighting their reduced robustness to geometric transformations such as flipping and rotation.

In contrast, the CNN-based method exhibits a steep rise near the origin and stays close to the upper-left corner in both subsets. This indicates strong discriminative capability even at very low false positive rates, underscoring the superior robustness of learned deep embeddings in the presence of high visual diversity and complex transformations in the ABO dataset.



(**a**) (**b**)

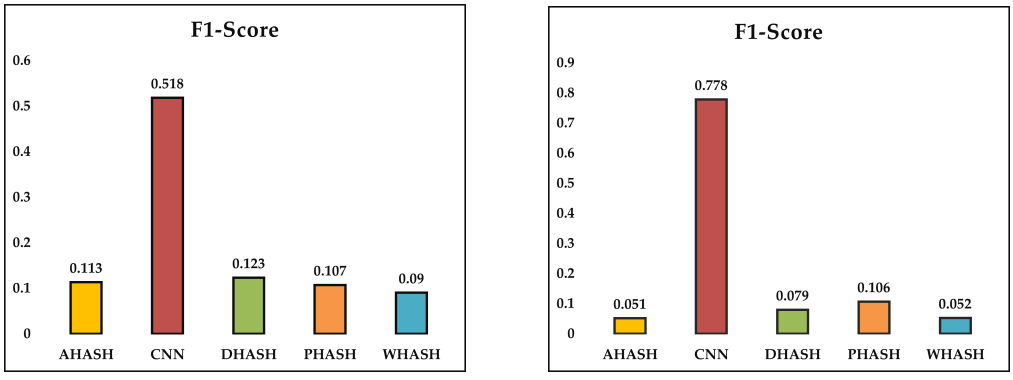
Figure 12. ROC curves for the ABO dataset: (**a**) Near Duplicate subset; (**b**) Transformed subset.

3.3.3. F1-Score Analysis (ABO)

Figure 13 summarizes the F1-score results for the ABO dataset. For the Near-Duplicate subset (Figure 13a), all perceptual hashing methods produce low F1-scores, showing that they struggle to balance precision and recall in the presence of high visual variability. Among the hashing approaches, DHash performs slightly better than the others, followed by AHash, PHash, and WHash, although the overall differences are small and the scores remain low.

A similar pattern is observed for the Transformed subset in Figure 13b. Under stronger geometric and appearance changes, such as rotation and aspect-ratio modification, the F1-scores of all hashing methods drop further, indicating very limited robustness.

In contrast, the CNN-based method achieves much higher F1-scores in both subsets, with particularly strong performance on the transformed images. This highlights that learned deep embeddings are far more effective than perceptual hashing methods at preserving discriminative information in visually diverse and heavily transformed product images.



(**a**) (**b**)

Figure 13. F1-score comparison for the ABO dataset using 100 randomly selected images: (**a**) Near Duplicate subset; (**b**) Transformed subset.

3.4. Quantitative Evaluation on the UKB Dataset

This subsection summarizes UKB quantitative results on the complete Exact, Near Duplicate, and Transformed subsets. To ensure comparability, each method is evaluated using three threshold settings: Hamming distance thresholds of 0, 10, and 32 for perceptual hashes and cosine similarity thresholds of 0.5, 0.9, and 1.0 for the CNN-based approach. These thresholds allow for direct comparisons of MAP, NDCG, Jaccard Index, and runtime

among all five approaches and the reason behind choosing these values are to simulate three real-world scenarios: Strict/Exact Matching (Hamming 0/CNN 1.0) for finding identical files; Moderate Matching (Hamming 10/CNN 0.9) for finding near-duplicates with minor changes; and Relaxed Matching (Hamming 32/CNN 0.5) to test the limits of the models against heavy transformations. The value of 32 for hashing represents a 50% bit-flip in the 64-bit hash, serving as a logical boundary for testing model failure.

3.4.1. Mean Average Precision (MAP) Results on the UKB Dataset

Table 1 summarizes MAP results for the three UKB subsets with the chosen thresholds. On the Near Duplicate subset, the hashing methods produce very low MAP at the strict threshold, but improve at the moderate threshold, with AHash and WHash slightly outperforming DHash and PHash. In contrast, the CNN approach achieves a much larger gain, with a MAP of 0.4182 at the moderate threshold, indicating significantly higher ranking quality.

Among the hashing techniques, PHash achieves the highest MAP for the Transformed subset at the moderate threshold; however, the absolute values are still low, suggesting poor robustness to geometric distortions. Once more, the CNN approach outperforms all other approaches at the mid-range threshold, reaching 0.6080 MAP.

For Exact Duplicates, all hashing methods under the strict threshold produce near-perfect MAP, consistent with identical image content. When the threshold is loosened, MAP falls significantly. In contrast, the CNN method achieves 0.9958 MAP at the moderate threshold and maintains moderate ranking performance even in the relaxed condition, indicating greater stability across duplicate types.

Table 1. MAP scores for the five methods on the UKB dataset under different threshold settings.

Dataset	Metric	Threshold	AHash	DHash	PHash	WHash	CNN
Near Duplicate	MAP	0/0.5	0.0014	0.0000	0.0001	0.0016	0.0104
		10/0.9	0.0412	0.0203	0.0161	0.0351	0.4182
		32/1.0	0.0028	0.0025	0.0021	0.0027	0.0000
Transformed	MAP	0/0.5	0.0626	0.0536	0.0577	0.0638	0.0152
		10/0.9	0.0761	0.0705	0.0801	0.0624	0.6080
		32/1.0	0.0028	0.0030	0.0027	0.0028	0.0075
Exact Duplicate	MAP	0/0.5	0.9989	1.0000	1.0000	0.9982	0.0112
		10/0.9	0.5190	0.8503	0.9944	0.4671	0.9958
		32/1.0	0.0037	0.0031	0.0031	0.0032	0.5851

3.4.2. Normalized Discounted Cumulative Gain (NDCG) Results on the UKB Dataset

The NDCG scores on UKB dataset are listed in Table 2, and they mostly follow the MAP trends. Hashing-based techniques yield very low NDCG for Near Duplicates at the strict threshold, followed by a moderate improvement at the mid-range setting. The CNN model shows a substantially larger gain at the moderate threshold, reflecting improved ranking performance under mild visual variation.

The NDCG for the hashing techniques is marginally higher for the Transformed subset than in the Near Duplicate setting, but the gains are still small and differ depending on the method. Once again, the CNN approach leads at the mid-range threshold, indicating that it more consistently preserves the ranking structure under geometric transformations.

For Exact Duplicates, all hashing techniques achieve near-perfect NDCG at the strict threshold, which is expected for identical images. Their scores significantly drop when the threshold is loosened, but the CNN stays powerful at the moderate setting. Overall, these NDCG results support the MAP trend: CNN embeddings offer more consistent ranking

across both minor and significant variations, while perceptual hashes perform best for identical content.

Table 2. NDCG scores for the five methods on the UKB dataset under different threshold settings.

Dataset	Metric	Threshold	AHash	DHash	PHash	WHash	CNN
Near Duplicate	NDCG	0/0.5	0.0036	0.0000	0.0004	0.0039	0.1539
		10/0.9	0.1380	0.0637	0.0429	0.1280	0.6666
		32/1.0	0.1273	0.1219	0.1203	0.1265	0.0000
Transformed	NDCG	0/0.5	0.3020	0.2656	0.2887	0.3122	0.1773
		10/0.9	0.2728	0.3053	0.3356	0.2435	0.7961
		32/1.0	0.1320	0.1356	0.1346	0.1322	0.0376
Exact Duplicate	NDCG	0/0.5	0.9992	1.0000	1.0000	0.9986	0.1249
		10/0.9	0.6156	0.8829	0.9958	0.5711	0.9968
		32/1.0	0.1040	0.1013	0.1010	0.1029	0.5851

The Jaccard Index results for UKB are shown in Table 3, which exhibits the same general pattern as MAP and NDCG. The hashing techniques yield extremely low Jaccard values on the Near Duplicate subset at the strict threshold and only slightly improve at the mid-level setting. The scores show little overlap between the retrieved sets and the actual duplicate sets, even at their best. The CNN approach, on the other hand, shows a much more dependable match between predicted and ground-truth duplicates and increases significantly at the moderate threshold.

For transformed duplicates, Jaccard scores for hashing methods rise slightly but remain low, indicating a lack of tolerance for geometric changes. The CNN model continues to outperform all others, with the highest overlap at the mid-level threshold and a clear advantage in transformed duplicate detection.

Table 3. Jaccard Index scores for the five methods on the UKB dataset under different threshold settings.

Dataset	Metric	Threshold	AHash	DHash	PHash	WHash	CNN
Near Duplicate	Jaccard	0/0.5	0.0013	0.0000	0.0001	0.0013	0.0036
		10/0.9	0.0324	0.0192	0.0160	0.0263	0.4167
		32/1.0	0.0005	0.0004	0.0004	0.0005	0.0000
Transformed	Jaccard	0/0.5	0.0625	0.0536	0.0577	0.0637	0.0063
		10/0.9	0.0714	0.0712	0.0803	0.0590	0.6072
		32/1.0	0.0007	0.0006	0.0006	0.0006	0.0075
Exact Duplicate	Jaccard	0/0.5	0.9980	1.0000	1.0000	0.9972	0.0023
		10/0.9	0.3945	0.8011	0.9892	0.3489	0.9913
		32/1.0	0.0004	0.0003	0.0003	0.0003	0.5851

For exact duplicates, the hashing methods achieve near-perfect Jaccard at the strict threshold, as expected given that the images are identical. When the threshold is relaxed, their scores drop dramatically, whereas the CNN remains strong at the moderate setting. Overall, these findings support the same conclusion: hashing is most reliable for truly identical images, whereas CNN has a more consistent overlap with the ground truth across all UKB duplicate conditions. It should be noted that certain scores (such as 1.0 or 0.0) in Tables 1–3 are the same. This is not accidental; rather, it is a reflection of the algorithms' mathematical structure. All properly operating hashes provide identical bits for exact duplicates, yielding a score of 1.0. Since hashing techniques are made to be sensitive to any pixel-level change, they all automatically score 0.0 for near-duplicates at a threshold of 0. When competing approaches find precisely the same number of accurate pairs from the fixed ground-truth set at a high threshold, they get identical fractional scores (like 0.5851).

3.4.3. Runtime Results (UKB)

Table 4 summarizes the runtime breakdowns for UKB dataset, including encoding, searching, and total time. Hashing methods have consistently low runtimes across all three subsets, ranging from 40 to 60 s. AHash is the most efficient, while WHash has a low overhead due to its wavelet-based processing.

In contrast, the CNN-based method has a significantly longer runtime. For the Near Duplicate and Transformed subsets, its feature-extraction (encoding) time is more than 90 s, and its overall runtime is roughly 240 s (four to six times longer than the hash-based approaches). The CNN is still noticeably slower than the handcrafted methods, even on the Exact Duplicate subset.

Overall, these findings demonstrate a clear trade-off: although CNN consistently yields higher accuracy, this comes at the cost of significantly longer computation times when compared to lightweight hashing methods.

Table 4. Runtime results (encoding + search time) on the UKB dataset.

Dataset	Method	Encode Time (Seconds)	Search Time (Seconds)	Total Time (Seconds)
Near Duplicate	AHash	8.875	30.932	39.807
	DHash	8.230	41.750	49.980
	PHash	9.157	40.558	49.715
	WHash	13.757	39.948	53.705
	CNN	98.712	141.138	239.850
Transformed	AHash	8.287	34.452	42.739
	DHash	7.257	48.260	55.517
	PHash	8.120	44.735	52.855
	WHash	12.918	44.897	57.815
	CNN	97.755	149.448	247.203
Exact Duplicate	AHash	4.364	7.965	12.329
	DHash	4.057	10.824	14.881
	PHash	4.498	10.444	14.942
	WHash	6.838	10.355	17.193
	CNN	46.394	35.683	82.077

3.5. Quantitative Evaluation on the ABO Dataset

Quantitative results for each of the five approaches using the ABO dataset are shown in this subsection. We assess each method at three fixed operating points to match the UKB setup and maintain fair comparisons: cosine similarity thresholds of 0.5, 0.9, and 1.0 for the CNN model, and Hamming distance thresholds of 0, 10, and 32 for the hashing methods. Direct comparison of MAP, NDCG, Jaccard Index, and runtime under the increased visual variability of ABO product images is made possible by these shared settings.

3.5.1. Mean Average Precision (MAP) Results on the ABO Dataset

The MAP results for ABO across the Exact Duplicate, Near Duplicate, and Transformed subsets are compiled in Table 5. The hashing techniques only slightly improve on the Near Duplicate subset when the threshold is loosened; AHash and PHash are slightly higher at the mid-level setting, but overall MAP values stay low because of the significant appearance variability in ABO product images. The CNN approach, on the other hand, exhibits a more pronounced increase at the moderate threshold (roughly 0.32), indicating that deep embeddings are better at ranking visually similar products in this dataset.

A similar pattern follows in the Transformed subset. Even with relaxed thresholds, the hashing methods produce low MAP values because common ABO edits (such as flips

and aspect-ratio changes) frequently break the handcrafted descriptors. The CNN approach outperforms, with a stronger rise at the moderate threshold (approximately 0.44), indicating greater resilience to geometric variation.

For Exact Duplicates, hashing methods produce high MAP under the strict threshold, as expected for truly identical images, but performance suffers significantly when the threshold is relaxed. In contrast, the CNN model achieves a significantly higher MAP at the mid-level threshold (approximately 0.91) while maintaining relatively stable ranking performance under the relaxed condition. Taken together, the ABO findings mirror UKB results, where CNN-based embeddings outperform hashing methods for ranking, especially when duplicates have higher appearance diversity or geometric transformations.

Table 5. MAP scores for the five methods on the ABO dataset under different threshold settings.

Dataset	Metric	Threshold	AHash	DHash	PHash	WHash	CNN
Near Duplicate	MAP	0/0.5	0.0217	0.0082	0.0060	0.0155	0.0158
		10/0.9	0.0591	0.0521	0.0903	0.0496	0.3159
		32/1.0	0.0058	0.0051	0.0052	0.0050	0.0003
Transformed	MAP	0/0.5	0.0505	0.0331	0.0298	0.0426	0.0278
		10/0.9	0.0452	0.0657	0.0526	0.0356	0.4446
		32/1.0	0.0049	0.0072	0.0063	0.0048	0.0000
Exact Duplicate	MAP	0/0.5	0.8769	0.9783	0.9831	0.8532	0.0220
		10/0.9	0.3100	0.7425	0.8965	0.2801	0.9084
		32/1.0	0.0056	0.0062	0.0063	0.0053	0.5899

3.5.2. Normalized Discounted Cumulative Gain (NDCG) Results on the ABO Dataset

Table 6 summarizes the NDCG performance on the ABO dataset. For Near Duplicates, hash-based methods produce low NDCG at the strict threshold and only modest gains at the moderate setting. The CNN approach results in a significant improvement (roughly 0.47), indicating higher ranking quality even for subtle visual differences between products.

A similar pattern is observed in the Transformed subset. The hash-based methods improve only slightly as thresholds are relaxed, indicating their sensitivity to common ABO edits like flips and other geometric changes. The CNN performs best at the moderate threshold (approximately 0.69), indicating better preservation of ranking quality during transformation.

For Exact Duplicates, hash-based methods produce near-perfect NDCG under the strict threshold, followed by a sharp decline as the threshold rises. In contrast, the CNN approach achieves a high mid-level score (approximately 0.93) and is more stable under relaxed conditions. Consistent with the MAP analysis, these findings show that perceptual hashes are most effective for truly identical images, whereas CNN embeddings maintain more reliable ranking across minor and major variations.

Table 6. NDCG scores for the five methods on the ABO dataset under different threshold settings.

Dataset	Metric	Threshold	AHash	DHash	PHash	WHash	CNN
Near Duplicate	NDCG	0/0.5	0.0527	0.0164	0.0120	0.0413	0.1585
		10/0.9	0.1671	0.1134	0.1442	0.1548	0.4726
		32/1.0	0.1261	0.1228	0.1255	0.1257	0.0007
Transformed	NDCG	0/0.5	0.1781	0.1242	0.1197	0.1675	0.2034
		10/0.9	0.1965	0.2367	0.2173	0.1750	0.6926
		32/1.0	0.1270	0.1479	0.1505	0.1259	0.0000
Exact Duplicate	NDCG	0/0.5	0.8994	0.9837	0.9873	0.8806	0.1456
		10/0.9	0.4295	0.7929	0.9214	0.3992	0.9303
		32/1.0	0.1140	0.1161	0.1158	0.1133	0.5929

3.5.3. Jaccard Index Results on the ABO Dataset

Table 7 reports the Jaccard Index values for ABO dataset. On the Near Duplicate subset, all hashing approaches produce very low scores under the strict threshold, with only minor gains at the moderate setting. PHash and DHash improve slightly more, but overall overlap remains low, implying that the retrieved sets correspond to only a small fraction of the ground-truth duplicates. The CNN model shows a more pronounced improvement at the mid-level threshold (approximately 0.30), indicating better duplicate retrieval under ABO's higher variability.

On the Transformed subset, Jaccard overlap for the hashing methods is low even at relaxed thresholds, indicating their sensitivity to flips, rotations, and aspect-ratio changes. The CNN again outperforms, reaching around 0.44 at the moderate threshold and maintaining a clear advantage in transformed duplicate detection.

Table 7. Jaccard Index scores for the five methods on the ABO dataset under different threshold settings.

Dataset	Metric	Threshold	AHash	DHash	PHash	WHash	CNN
Near Duplicate	Jaccard	0/0.5	0.0176	0.0081	0.0060	0.0142	0.0061
		10/0.9	0.0361	0.0451	0.0820	0.0324	0.3003
		32/1.0	0.0009	0.0008	0.0008	0.0008	0.0003
Transformed	Jaccard	0/0.5	0.0491	0.0328	0.0298	0.0407	0.0137
		10/0.9	0.0372	0.0629	0.0534	0.0299	0.4393
		32/1.0	0.0013	0.0016	0.0015	0.0013	0.0000
Exact Duplicate	Jaccard	0/0.5	0.8400	0.9581	0.9672	0.8134	0.0068
		10/0.9	0.2073	0.6733	0.8205	0.1926	0.8458
		32/1.0	0.0007	0.0007	0.0007	0.0007	0.5783

In the Exact Duplicate subset, all hashing methods achieve very high Jaccard overlap at the strict threshold, as expected given that identical images are easy to match. When the threshold is relaxed, their scores decrease significantly, while the CNN remains strong at a moderate setting (roughly 0.85). Overall, the Jaccard index confirms that hashes work best for perfect matches, whereas CNN retrieval remains more consistent across variations in the ABO dataset, which was also true for the UKB dataset.

3.5.4. Runtime Results (ABO)

Table 8 highlights runtime breakdowns (encoding, search, and total time)cfor ABO dataset. According to the UKB findings, hashing methods remain fast across all three subsets. Total runtime for Near Duplicate and Transformed images is usually in the 9–12 s range, with AHash and DHash being the most efficient and WHash incurring a minor overhead due to wavelet processing. The Exact Duplicate subset is faster overall, with total times typically less than 7 s.

In contrast, the CNN approach remains significantly more expensive. Encoding alone takes over 30 s on the Near Duplicate subset and more than 50 s on the Transformed subset, for a total runtime of approximately 46 and 65 s respectively. Even for Exact Duplicates, where hashing takes only a few seconds, the CNN is significantly slower.

Overall, the ABO runtime results reflect the same trade-off as the UKB, such that CNN embeddings provide much higher retrieval accuracy but require significantly more computation, whereas perceptual hashing remains much faster but with lower accuracy.

Table 8. Runtime results (encoding + search time) on the ABO dataset.

Dataset	Method	Encode Time (Seconds)	Search Time (Seconds)	Total Time (Seconds)
Near Duplicate	AHash	4.732	4.903	9.635
	DHash	4.591	4.425	9.016
	PHash	5.343	3.912	9.255
	WHash	6.397	4.777	11.174
	CNN	34.261	12.504	46.765
Transformed	AHash	7.776	3.444	11.220
	DHash	8.007	3.700	11.707
	PHash	8.586	3.800	12.386
	WHash	11.028	3.832	14.860
	CNN	53.210	12.103	65.313
Exact Duplicate	AHash	2.853	2.005	4.858
	DHash	3.063	1.892	4.955
	PHash	3.309	1.948	5.257
	WHash	4.351	2.272	6.623
	CNN	23.122	5.576	28.698

4. Discussion

For Exact Duplicate, Near Duplicate, and Transformed situations, Section 3 contrasts a CNN-based embedding model with traditional perceptual hashing techniques. Based on our findings, we conclude with a detailed analysis of the performance trade-offs and their implications for practical applications.

4.1. Performance Analysis of Hashing vs. CNN Paradigms

For exact-duplicate identification, perceptual hashing methods (AHash, DHash, PHash, and WHash) are quite effective. They remain extremely fast on both UKB and ABO while achieving near-perfect MAP, NDCG, and Jaccard under strict thresholds. This outcome is consistent with the basic goal of perceptual hashing, which is to employ compact signatures for speedy comparison, making these techniques practical for large-scale systems where identical images predominate.

However, when image variance rises, perceptual hashing's drawbacks become apparent. For Near Duplicate and Transformed situations, performance drastically declines, especially when geometric modifications like rotation or cropping are present. The CNN-based embedding approach, on the other hand, does well across the board. By capturing semantic structure rather than depending on low-level pixel data, it consistently outperforms hashing on Near Duplicate and Transformed images. The CNN pipeline offers an essential answer for unrestricted real-world circumstances, despite its slower processing speed.

4.2. Decision Framework for Users

To facilitate the selection of proper deduplication techniques in practical settings, we include a decision framework (Table 9) that maps our experimental results to specific industrial requirements. This framework serves as a guide for users to decide on the trade-offs between computational throughput and accurate detection.

Table 9. Decision framework for selecting an appropriate image matching method based on engineering requirements.

Engineering Requirement	Recommended Method	Evidence from Results
High-Speed Exact Matching	DHash/AHash	Achieved near-perfect Jaccard overlap (0.95–1.0) for exact duplicates while remaining significantly faster than CNN encoding.
Robustness to Geometric Changes (Rotation, Flipping)	CNN (VGG-16)	Maintained high F1-scores and MAP for both datasets, on the other hand, all hashing methods showed a degraded performance failure.
Near-Duplicate Detection (Lighting/Compression)	CNN/PHash	While CNN is the most reliable, PHash offers a workable "middle ground," retaining superior ranking metrics over other low-overhead hashes.
Large-Scale Web Crawling (High Throughput)	AHash/DHash	Most efficient methods tested; total processing times were significantly less compared to the much higher latency of the CNN pipeline.
High Precision in Diverse Datasets (e.g., ABO/Product images)	CNN (VGG-16)	Exhibited a better ROC rise and higher NDCG in visually complex datasets (ABO)when compared to the handcrafted hashing algorithms.

4.3. Standardized Similarity Metrics and Thresholds

For the perceptual hashing methods (AHash, DHash, PHash, and WHash), the degree of similarity S_{hash} is represented by the normalized Hamming distance calculation previously defined in Equation (12), where $S_{hash} = 1.0$ represents an exact duplicate and $S_{hash} = 0$ represents maximum divergence. Based on our experimental results, a threshold of $S_{hash} \geq 0.93$ is considered for identifying as near-duplicates with a high precision.

Similarly, using the CNN-based method, the Cosine Similarity given in Equation (11) is equivalent to the degree of similarity S_{cnn}. Our study shows that the CNN model maintains a high degree of similarity ($S_{cnn} > 0.85$) even under significant geometric alterations, when hashing techniques typically fail. Based on the UKBench and ABO data, we classify these scores into three categories:

- Exact or Near-Identical: $0.95 \leq S \leq 1.00$;
- Transformation or Near-Duplicate: $0.80 \leq S \leq 0.94$;
- Unrelated or Dissimilar: $S < 0.70$.

5. Conclusions and Future Work

This study offers a comprehensive empirical comparison of popular image deduplication techniques, such as CNN-based embedding models and traditional perceptual hashing (AHash, DHash, PHash, and WHash), assessed under uniform experimental conditions. Perceptual hashing techniques are very effective and accurate for exact-duplicate detection, but they drastically deteriorate in near-duplicate and transformed scenarios, according to results on the UKBench and Amazon Berkeley Objects datasets. Although the CNN-based embedding method offers significantly greater robustness, it comes at a higher computational cost. When combined, these findings shed light on the trade-off between robustness and efficiency in image deduplication and offer useful recommendations for method selection in real-world applications. To assist users in choosing the best course of action given their unique precision and latency restrictions, we developed a decision framework which offers an essential benchmark for industrial deduplication systems, where scalability depends on tool selection. In order to improve large-scale production environments, future work will expand this study to incorporate a wider variety of deep architectures, such

as transformer-based models, and explore two-stage hybrid pipelines that combine the resilience of deep learning with the speed of hashing.

Author Contributions: Conceptualization, W.D.P., M.F.M. and Z.N.; methodology, W.D.P., M.F.M. and Z.N.; software, M.F.M., Z.N. and W.D.P.; validation, M.F.M., Z.N. and W.D.P.; formal analysis, M.F.M., Z.N. and W.D.P.; investigation, M.F.M., Z.N. and W.D.P.; resources, M.F.M., Z.N. and W.D.P.; data curation, M.F.M.; writing—original draft preparation, M.F.M., Z.N. and W.D.P.; writing—review and editing, M.F.M., Z.N. and W.D.P.; visualization, M.F.M., Z.N. and W.D.P.; supervision, W.D.P.; project administration, W.D.P.; funding acquisition, W.D.P. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The datasets used in this study are publicly available. The UKBench (UKB) dataset can be accessed at https://archive.org/details/ukbench (accessed on 16 December 2025). The Amazon–Berkeley Objects (ABO) dataset is publicly available at https://amazon-berkeley-objects.s3.amazonaws.com/index.html (accessed on 16 December 2025).

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Rydning, J. *Worldwide IDC Global Datasphere Forecast, 2023–2027: It's a Distributed, Diverse, and Dynamic (3D) Datasphere*; IDC Market Forecast; International Data Corporation: Needham, MA, USA, 2023.
2. Cisco Systems, Inc. *Cisco Annual Internet Report (2018–2023)*; Technical Report; Cisco Systems, Inc.: San Jose, CA, USA, 2023.
3. Hilbert, M.; López, P. The World's Technological Capacity to Store, Communicate, and Compute Information. *Science* **2011**, *332*, 60–65. [CrossRef] [PubMed]
4. Bartley, K. Big Data Statistics: How Much Data Is There in the World? 2025. Available online: https://rivery.io/blog/big-data-statistics-how-much-data-is-there-in-the-world/ (accessed on 25 December 2025).
5. Kumar, N. Big Data Statistics 2026 (Growth, Trends & Market Size). 2025. Available online: https://www.demandsage.com/big-data-statistics/ (accessed on 31 December 2025).
6. International Data Corporation. *Worldwide IDC Global DataSphere Forecast Including 2022–2026*; International Data Corporation: Needham, MA, USA, 2025.
7. Barroso, L.A.; Clidaras, J.; Hölzle, U. *The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines*; Synthesis Lectures on Computer Architecture; Morgan & Claypool Publishers: San Rafael, CA, USA, 2013; Volume 8, pp. 1–154.
8. Khan, A.Q.; Matskin, M.; Prodan, R.; Bussler, C.; Roman, D.; Soylu, A. Cloud storage cost: A taxonomy and survey. *World Wide Web* **2024**, *27*, 36. [CrossRef]
9. El-Shimi, A.; Kalach, R.; Kumar, A.; Ottean, A.; Li, J.; Sengupta, S. Primary data {Deduplication—Large} scale study and system design. In Proceedings of the 2012 USENIX Annual Technical Conference (USENIX ATC 12), Boston, MA, USA, 13–15 June 2012; pp. 285–296.
10. Henzinger, M.R. Finding Near-Duplicate Web Pages: A Large-Scale Evaluation of Algorithms. In *Proceedings of the International World Wide Web Conference (WWW)*; Association for Computing Machinery: New York, NY, USA, 2006.
11. Mandagere, N.; Zhou, D.; Smith, K.; Uttamchandani, D. Demystifying Data Deduplication. In *Proceedings of the ACM/IFIP International Conference on Middleware*; ACM: New York, NY, USA, 2008; pp. 1–12.
12. Wang, X.J.; Zhang, L.; Liu, C. Duplicate discovery on 2 billion internet images. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*; IEEE Computer Society: Los Alamitos, CA, USA, 2013; pp. 429–436.
13. Thyagharajan, K.; Kalaiarasi, G. A review on near-duplicate detection of images using computer vision techniques. *Arch. Comput. Methods Eng.* **2021**, *28*, 897–916. [CrossRef]
14. Recht, B.; Roelofs, R.; Schmidt, L.; Shankar, V. Do imagenet classifiers generalize to imagenet? In Proceedings of the International Conference on Machine Learning, PMLR, Long Beach, CA, USA, 9–15 June 2019; pp. 5389–5400.
15. Wright, A. *Worldwide IDC Global Datasphere Forecast, 2024–2028: AI Everywhere, but Upsurge in Data Will Take Time*; International Data Corporation: Framingham, MA, USA, 2024.
16. Kaur, R.; Chana, I.; Bhattacharya, J. Data deduplication techniques for efficient cloud storage management: A systematic review. *J. Supercomput.* **2018**, *74*, 2035–2085. [CrossRef]
17. Venkatesan, R.; Koon, S.M.; Jakubowski, M.H.; Moulin, P. Robust image hashing. In *Proceedings 2000 International Conference on Image Processing (Cat. No. 00CH37101)*; IEEE: Piscataway, NJ, USA, 2000; Volume 3, pp. 664–666.

18. Porwik, P.; Lisowska, A. The Haar-wavelet transform in digital image processing: Its status and achievements. *Mach. Graph. Vis.* **2004**, *13*, 79–98.
19. Zauner, C. Implementation and Benchmarking of Perceptual Image Hash Functions. Ph.D. Thesis, University of Applied Sciences Hagenberg, Hagenberg, Austria, 2010.
20. Alkhowaiter, M.; Almubarak, K.; Zou, C. Evaluating perceptual hashing algorithms in detecting image manipulation over social media platforms. In *2022 IEEE International Conference on Cyber Security and Resilience (CSR)*; IEEE: Piscataway, NJ, USA, 2022; pp. 149–156.
21. Monga, V.; Evans, B.L. Perceptual image hashing via feature points: Performance evaluation and tradeoffs. *IEEE Trans. Image Process.* **2006**, *15*, 3452–3465. [CrossRef]
22. Swaminathan, A.; Mao, Y.; Wu, M. Robust and secure image hashing. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*; IEEE: Piscataway, NJ, USA, 2006.
23. Khelifi, F.; Jiang, J. Perceptual image hashing based on virtual watermark detection. *IEEE Trans. Image Process.* **2009**, *19*, 981–994. [CrossRef] [PubMed]
24. Sharma, S. Distance distributions and runtime analysis of perceptual hashing algorithms. *J. Vis. Commun. Image Represent.* **2024**, *104*, 104310. [CrossRef]
25. McKeown, S. Beyond Hamming Distance: Exploring Spatial Encoding in Perceptual Hashes. *Forensic Sci. Int. Digit. Investig.* **2025**, *52*, 301878. [CrossRef]
26. Kotzer, A.; Naamneh, M.; Rottenstreich, O.; Reviriego, P. Detection of NFT Duplications with Image Hash Functions. In *2024 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*; IEEE: Piscataway, NJ, USA, 2024; pp. 1–7.
27. Krizhevsky, A.; Sutskever, I.; Hinton, G.E. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*; Curran Associates, Inc.: Red Hook, NY, USA, 2012; Volume 25.
28. Simonyan, K.; Zisserman, A. Very deep convolutional networks for large-scale image recognition. In Proceedings of the International Conference on Learning Representations, San Diego, CA, USA, 7–9 May 2015.
29. Wang, X.; Zhao, Y.; Pourpanah, F. Recent advances in deep learning. *Int. J. Mach. Learn. Cybern.* **2020**, *11*, 747–750. [CrossRef]
30. Xia, W.; Jiang, H.; Feng, D.; Douglis, F.; Shilane, P.; Hua, Y.; Fu, M.; Zhang, Y.; Zhou, Y. A comprehensive study of the past, present, and future of data deduplication. *Proc. IEEE* **2016**, *104*, 1681–1710. [CrossRef]
31. Truong, T.; Jush, F.K.; Lenga, M. Benchmarking pretrained vision embeddings for near-and duplicate detection in medical images. In *2024 IEEE International Symposium on Biomedical Imaging (ISBI)*; IEEE: Piscataway, NJ, USA, 2024; pp. 1–5.
32. Jakhar, Y.; Borah, M.D. Effective near-duplicate image detection using perceptual hashing and deep learning. *Inf. Process. Manag.* **2025**, *62*, 104086. [CrossRef]
33. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems*; Curran Associates, Inc.: Red Hook, NY, USA, 2017; Volume 30.
34. Chen, Y.; Zhang, S.; Liu, F.; Chang, Z.; Ye, M.; Qi, Z. Transhash: Transformer-based hamming hashing for efficient image retrieval. In *Proceedings of the 2022 International Conference on Multimedia Retrieval*; Association for Computing Machinery: New York, NY, USA, 2022; pp. 127–136.
35. Mahmud, M.F.; Nusrat, Z.; Pan, W.D. Lossless Compression of Malaria-Infected Erythrocyte Images Using Vision Transformer and Deep Autoencoders. *Computers* **2025**, *14*, 127. [CrossRef]
36. Wang, J.; Zhang, T.; Song, J.; Sebe, N.; Shen, H.T. A survey on learning to hash. *IEEE Trans. Pattern Anal. Mach. Intell.* **2017**, *40*, 769–790. [CrossRef]
37. Suryawanshi, D. Image Recognition: Detection of Nearly Duplicate Images. Ph.D. Thesis, California State University Channel Islands, Camarillo, CA, USA, 2018.
38. Hamadouche, M.; Zebbiche, K.; Guerroumi, M.; Tebbi, H.; Zafoune, Y. A comparative study of perceptual hashing algorithms: Application on fingerprint images. In Proceedings of the 2nd International Conference on Computer Science's Complex Systems and their Applications, Oum El Bouaghi, Algeria, 25–26 May 2021.
39. Hao, Q.; Luo, L.; Jan, S.T.; Wang, G. It's not what it looks like: Manipulating perceptual hashing based applications. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*; Association for Computing Machinery: New York, NY, USA, 2021; pp. 69–85.
40. Ahmed, N.; Natarajan, T.; Rao, K. Discrete cosine transform. *IEEE Trans. Comput.* **1974**, *C-23*, 90–93. [CrossRef]
41. Wallace, G.K. The JPEG still picture compression standard. *Commun. ACM* **1991**, *34*, 30–44. [CrossRef]
42. Nusrat, Z.; Mahmud, M.F.; Pan, W.D. Efficient Compression of Red Blood Cell Image Dataset Using Joint Deep Learning-Based Pattern Classification and Data Compression. *Electronics* **2025**, *14*, 1556. [CrossRef]
43. Li, X.; Qin, C.; Wang, Z.; Qian, Z.; Zhang, X. Unified performance evaluation method for perceptual image hashing. *IEEE Trans. Inf. Forensics Secur.* **2022**, *17*, 1404–1419. [CrossRef]
44. Stanković, R.S.; Falkowski, B.J. The Haar wavelet transform: Its status and achievements. *Comput. Electr. Eng.* **2003**, *29*, 25–44. [CrossRef]

45. Jain, T.; Lennan, C.; John, Z.; Tran, D. Imagededup. 2019. Available online: https://github.com/idealo/imagededup (accessed on 14 October 2025).
46. Babenko, A.; Slesarev, A.; Chigorin, A.; Lempitsky, V. Neural codes for image retrieval. In *Computer Vision—ECCV 2014*; European Conference on Computer Vision; Springer: Cham, Switzerland, 2014; pp. 584–599.
47. Ahmed, T.; Das, P.; Ali, M.F.; Mahmud, M.F. A comparative study on convolutional neural network based face recognition. In *2020 11th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*; IEEE: Piscataway, NJ, USA, 2020; pp. 1–5.
48. Lin, M.; Chen, Q.; Yan, S. Network in Network. *arXiv* **2014**, arXiv:1312.4400.
49. Chen, Y.; Jiang, H.; Li, C.; Jia, X.; Ghamisi, P. Deep feature extraction and classification of hyperspectral images based on convolutional neural networks. *IEEE Trans. Geosci. Remote Sens.* **2016**, *54*, 6232–6251. [CrossRef]
50. Sharif Razavian, A.; Azizpour, H.; Sullivan, J.; Carlsson, S. CNN features off-the-shelf: An astounding baseline for recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*; IEEE Computer Society: Los Alamitos, CA, USA, 2014; pp. 806–813.
51. Manning, C.D.; Raghavan, P.; Schütze, H. *Introduction to Information Retrieval*; Cambridge University Press: Cambridge, UK, 2008.
52. Powers, D.M.W. Evaluation: From Precision, Recall and F-Score to ROC, Informedness, Markedness and Correlation. *J. Mach. Learn. Technol.* **2011**, *2*, 37–63.
53. Hosny, K.M.; Hamza, H.M.; Lashin, N.A. Copy-for-duplication forgery detection in colour images using QPCETMs and sub-image approach. *IET Image Process.* **2019**, *13*, 1437–1446. [CrossRef]
54. Fawcett, T. An Introduction to ROC Analysis. *Pattern Recognit. Lett.* **2006**, *27*, 861–874. [CrossRef]
55. Rainio, O.; Teuho, J.; Klén, R. Evaluation metrics and statistical tests for machine learning. *Sci. Rep.* **2024**, *14*, 6086. [CrossRef]
56. Sasaki, Y. The Truth of the F-Measure. Available online: https://people.cs.pitt.edu/~litman/courses/cs1671s20/F-measure-YS-26Oct07.pdf (accessed on 29 March 2026).
57. Yilmaz, E.; Aslam, J.A. Estimating average precision when judgments are incomplete. *Knowl. Inf. Syst.* **2008**, *16*, 173–211. [CrossRef]
58. Järvelin, K.; Kekäläinen, J. Cumulated gain-based evaluation of IR techniques. *ACM Trans. Inf. Syst.* **2002**, *20*, 422–446. [CrossRef]
59. Burges, C.; Shaked, T.; Renshaw, E.; Lazier, A.; Deeds, M.; Hamilton, N.; Hullender, G. Learning to Rank using Gradient Descent. In *Proceedings of the International Conference on Machine Learning (ICML)*; Association for Computing Machinery: New York, NY, USA, 2005; pp. 89–96.
60. Wang, Y.; Wang, L.; Li, Y. A theoretical analysis of NDCG ranking measures. *Inf. Retr. J.* **2013**, *16*, 160–182.
61. Jaccard, P. Étude Comparative de la Distribution Florale dans une Portion des Alpes et des Jura. *Bull. Soc. Vaudoise Sci. Nat.* **1901**, *37*, 547–579.
62. Tan, P.N.; Steinbach, M.; Kumar, V. Introduction to Data Mining. In *Foundations of SQL Server 2008 R2 Business Intelligence*; Apress: Berkeley, CA, USA, 2006.
63. Jégou, H.; Douze, M.; Schmid, C. Product Quantization for Nearest Neighbor Search. *IEEE Trans. Pattern Anal. Mach. Intell.* **2011**, *33*, 117–128. [CrossRef]
64. Gordo, A.; Almazán, J.; Revaud, J.; Larlus, D. Deep Image Retrieval: Learning Global Representations for Image Search. In *Computer Vision—ECCV 2016*; Springer: Cham, Switzerland, 2016; pp. 241–257.
65. Tolias, G.; Sicre, R.; Jégou, H. Particular Object Retrieval with Integral Max-Pooling of CNN Activations. In Proceedings of the International Conference on Learning Representations (ICLR), San Juan, PR, USA, 2–4 May 2016.
66. Nistér, D.; Stewénius, H. Scalable Recognition with a Vocabulary Tree. In *2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*; IEEE: Piscataway, NJ, USA, 2006; pp. 2161–2168. [CrossRef]
67. Collins, J.; Goel, S.; Deng, K.; Luthra, A.; Xu, L.; Gundogdu, E.; Zhang, X.; Yago Vicente, T.F.; Dideriksen, T.; Arora, H.; et al. ABO: Dataset and Benchmarks for Real-World 3D Object Understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*; IEEE: Piscataway, NJ, USA, 2022.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.